
Flipdeck

Fix & Flip Portfolio Manager

Foundational Real Estate

Document	Build log, operations runbook, and user guide
Live application	https://flipdeckapp.vercel.app
Source	github.com/arahfranco/flipdeckapp
Prepared	22 July 2026
Covers	38 commits, 7 database migrations, 69 automated tests

This document has three parts. Part 1 records what was built and why the unusual decisions were made. Part 2 is the runbook for keeping the live system running. Part 3 explains how to use the application day to day.

Contents

Part	Section	Covers
1	What Flipdeck is	Purpose and shape of the system
1	Architecture	Technology choices and the reason for each
1	The rollup chain	The central calculation everything depends on
1	The five categories	Budget structure and subcategories
1	Decisions that shaped the build	Money handling, derived values, role design
1	Bank import: three real bugs	Statement quirks the importer must survive
1	Build timeline	Phases and what each delivered
1	Database migrations	Every schema change, in order
1	Testing	What is covered and what is not
2	Services	The four external accounts and their roles
2	Environment variables	Every setting, where it lives, what breaks
2	The quote trap	The single most costly configuration mistake
2	Deploying a change	Normal flow and the build-time rule
2	Running a database migration	Safe procedure against live data
2	Adding a user	Invite-only access model
2	Troubleshooting	Symptom, cause and fix for known failures
3	Signing in	Magic-link login
3	What each role can see	Permission matrix
3	Properties	Creating, editing, photos, status
3	Budget	Estimates and where Actual comes from
3	Expenses	Logging spend and attaching receipts
3	Money In and Out	Bank accounts, CSV import, posting
3	Payroll and workers	Hours, rates and the Labor line
3	Partners and capital	Equity, loans and draws
3	Company Value	Reading the balance sheet
3	Settings	Company details and user accounts

PART 1

Build & Decisions Log

What was built, in what order, and the reasoning behind the choices that are not obvious from reading the code.

What Flipdeck is

Flipdeck is a multi-user web application for running a fix-and-flip property business. It tracks each property from scouting through to sale: the budget, what was actually spent, the labour, the money moving through the bank, the capital partners behind it, and what the company as a whole is worth.

It was rebuilt from a working single-file browser prototype into a real server application with accounts, permissions, and a database. The prototype's calculation logic was carried across deliberately intact, because it encoded business rules that had already been proven in use.

Architecture

Layer	Choice	Why this one
Framework	Next.js 14 (App Router)	Server components mean financial data is calculated on the server and never shipped to the browser as raw numbers.
Database	PostgreSQL on Neon	Managed, serverless-friendly connection pooling, generous free tier.
Data access	Prisma 5	Typed schema, and migrations that can be reviewed as SQL before touching live data.
Money type	Decimal(12,2)	Never floating point. See the note below.
Authentication	NextAuth v5, email links	No passwords to store, leak, or reset. Sessions live in the database.
Email	Resend HTTP API	SMTP connections are unreliable on serverless hosting; an HTTP call is not.
File storage	Cloudflare R2	S3-compatible, no egress charges.
Hosting	Vercel	Deploys automatically on push to the main branch.
Tests	Vitest	69 unit tests over the calculation and import logic.

Money is never a floating point number.

Every monetary column is Decimal(12,2) and every calculation runs through Decimal arithmetic end to end. Floating point cannot represent 0.10 exactly, so repeated addition drifts. On a portfolio adding thousands of line items, that drift becomes a balance sheet that does not reconcile, with no single wrong entry to find.

The rollup chain

This is the calculation the entire application depends on. It lives in lib/calc.ts and is the most heavily tested code in the project.

Step	What happens
1. Expenses are summed by subcategory	Every expense in the log is grouped by its subcategory.
2. Payroll becomes a Labor line	Hours multiplied by rate across all payroll entries, added to Rehab Costs as a synthetic 'Labor (Payroll)' row that cannot be edited directly.
3. Actual overrides the budget	Where expenses exist for a subcategory, that sum becomes the Actual figure. It is derived, never stored, so it cannot go stale.
4. Categories roll up	Each of the five categories totals its estimated and actual columns.
5. Cost and price are compared	Total actual cost against the price basis gives profit and margin.

Two rules inside that chain are easy to get backwards

The price basis switch.

Until a property sells, actual profit is measured against the **estimated** sale price. Once a real sale price exists, it is measured against that. Getting this backwards makes every unsold property display a catastrophic loss, because it would be comparing real costs against a sale price of zero.

Which categories accept a typed-in Actual.

Purchase Costs and Selling Price are entered once, from a closing statement. Their Actual figures are typed in and kept. Every other cost category is purely expense-derived and reads zero until a real expense exists. This was a deliberate correction: previously a stored Actual could survive after a property was repurposed, showing spending that no longer applied to anything.

The five categories

Four are costs. The fifth, Selling Price, is revenue and is treated separately everywhere in the calculation.

Category	Subcategories
Purchase Costs	Purchase Price, Closing Costs, Title & Escrow, Inspection, Appraisal, Transfer Tax
Rehab Costs	33 lines covering structure, systems and finishes: Demolition, Foundation, Framing, Roofing, Siding, Windows, Exterior Doors, Gutters, Electrical, Plumbing, HVAC, Insulation, Drywall, Interior Doors, Trim & Millwork, Interior Paint, Exterior Paint, Flooring, Tile, Kitchen Cabinets, Countertops, Appliances, Bathroom Fixtures, Lighting, Hardware, Landscaping, Fencing, Driveway & Concrete, Permits & Inspections, Dumpster & Cleanup, Building Materials, Miscellaneous and Permits, Contingency, plus the derived Labor (Payroll) line

Category	Subcategories
Holding Costs	Loan Interest, Property Tax, Insurance, Utilities, HOA Dues, Security & Maintenance
Selling Costs	Listing Commission, Buyer Commission, Staging, Photography, Escrow Fees, Seller Concessions
Selling Price	Sale Price

Adding a subcategory is not just a list edit.

The rollup is built from the budget lines that exist on a property. An expense whose subcategory has no matching budget line is dropped from the category total silently, with no error and no visible gap. New subcategories must therefore be backfilled onto existing properties, or spending logged against them would understate cost and overstate profit.

Decisions that shaped the build

Roles are deliberately asymmetric

The permission model is not a simple ladder from least to most access. A Bookkeeper can see payroll but not partner capital. A Partner can see capital but not payroll. This is intentional: the people reconciling the books have no business seeing equity positions, and investors have no business seeing individual wages.

Permissions are enforced on the server in every API route, not by hiding navigation links. Hidden links are a convenience; they are not security.

Cash on hand is derived, not typed in

Bank transactions were originally one-directional, recording only money going out. That made cash impossible to calculate, so the first version of the balance sheet used hand-maintained cash balances that could drift from reality.

Transactions now carry a direction. Money out posts to an expense; money in posts to property income. An account's balance is its opening balance plus money in minus money out, calculated fresh every time. Only the opening balance is editable, so the displayed figure cannot disagree with the transactions underneath it.

A sold property leaves the balance sheet

Company assets count held property value plus cash. A sold property is excluded, because its proceeds have already become cash. Counting both would double-count the same money.

Partner loans are debt; partner equity is not

Capital contributions come in three kinds. EQUITY is owner capital. DRAW is capital taken back out, subtracted from equity. LOAN is money the company genuinely owes and is reported under liabilities, listed separately from third-party debt so the distinction stays visible.

Rental income counts only Rented properties

The Rented status was added for the buy-and-hold exit, where a property is kept and let out rather than listed. Rental income on the dashboard counts only properties in that status, so a rent figure entered on a property still in rehab reads as the projection it is, rather than money being collected.

Bank import: three real bugs it must survive

These behaviours look like quirks and are load-bearing. They come from real bank exports and are covered by tests so they cannot be tidied away.

Problem	What happens without the fix
Amex writes charges as positive numbers; Chase writes them as negative.	The importer detects which convention a file uses. Guessing wrong does not merely skip rows: now that deposits are imported too, it would reverse them, turning spending into income.
Chase statements contain a 'Details' column that collides with 'Description'.	Column detection follows a strict alias precedence. Without it, the importer reads the wrong column and every transaction description is meaningless.
The same statement may be imported twice, or into two accounts.	Each row gets a fingerprint of account, date, amount, direction and description. Same account and same fingerprint means duplicate. Including direction stops a deposit cancelling out a withdrawal of the same date and amount; including account stops one bank's statement blocking another's.

Build timeline

Phase	Delivered
Foundations	Project scaffold, database schema, the rollup calculation ported from the prototype with unit tests, magic-link authentication, server-side role checks.
Core screens	Portfolio overview, property list and detail, budget, expenses, payroll, partners.
Editing	Add, edit and delete across expenses, payroll, contributions, workers, bank transactions and properties. Cascading deletes with an impact preview.
Bank reconciliation	CSV import with column mapping, duplicate detection and category suggestions; posting a transaction to a property.
Files	Receipt and property photo uploads to Cloudflare R2.
Going live	Deployment to Vercel, custom email delivery through Resend, invite-only sign-in, user management.
Company Value	Balance sheet dashboard: property values, liabilities, cash, partner capital, company equity.
Money in and out	Bi-directional bank transactions, real bank accounts, property income, and cash on hand derived from the transactions.
Refinements	Rented status, rental income scoped to it, two new rehab subcategories backfilled onto existing properties.

Database migrations

Every schema change is a reviewed SQL file. Changes against live data are written additively: add the new shape, backfill it, then tighten constraints, so an existing row is never left invalid mid-migration.

Migration	What it did
init	Initial schema.

Migration	What it did
add_worker_entity	Workers became a real table instead of free text on payroll.
finalize_worker_relation	Second step: made the worker link required after backfill.
company_balance_sheet	Market value and monthly rent on properties; liabilities.
bank_money_in_out	Direction on transactions, real bank accounts, property income. Three steps with a backfill that rewrote every transaction fingerprint.
property_status_rented	Added the Rented status.
rehab_materials_misc_lines	Added two rehab subcategories and created the budget lines on existing properties.

Testing

69 unit tests cover the parts where a silent error would be expensive and invisible: the rollup chain, the balance sheet, CSV import including the sign and column bugs above, and configuration reading.

What the tests do not cover.

There are no end-to-end browser tests. Permission enforcement, the upload path and the sign-in flow are verified by hand. A regression in those would not be caught automatically.

PART 2

Setup & Operations Runbook

How the live system is wired together, and what to do when part of it stops working.

Services

Service	Holds	If it goes down
Neon	The PostgreSQL database. All business data and login sessions.	The application cannot load anything.
Vercel	The application itself. Rebuilds and redeploys on every push to main.	The site is unreachable.
Cloudflare R2	Uploaded files: property photos, receipts, company logo.	Uploads fail; existing images stop displaying.
Resend	Delivers the sign-in emails.	Nobody new can sign in; existing sessions keep working.
GitHub	Source code. Pushing to main triggers a deployment.	No deployments, but the live site keeps running.

Environment variables

These live in two places: a local `.env` file for development, and Vercel's settings for the live site. The `.env` file is deliberately excluded from GitHub, which means Vercel never receives it automatically. Anything added locally must be added to Vercel by hand.

Variable	Purpose
<code>DATABASE_URL</code>	Connection string for the Neon database.
<code>DIRECT_URL</code>	Direct (unpooled) connection, used for migrations.
<code>AUTH_SECRET</code>	Signs session tokens. The application returns a server error without it.
<code>NEXTAUTH_SECRET</code>	Same value, under the older name. Both are set.
<code>NEXTAUTH_URL</code>	The site's own address.
<code>EMAIL_SERVER_HOST</code>	<code>smtp.resend.com</code>
<code>EMAIL_SERVER_PORT</code>	<code>587</code>
<code>EMAIL_SERVER_USER</code>	<code>resend</code>
<code>EMAIL_SERVER_PASSWORD</code>	The Resend API key. Begins with <code>re_</code> and is sent over HTTPS, not SMTP.
<code>EMAIL_FROM</code>	The sender address shown on sign-in emails.

Variable	Purpose
R2_ACCOUNT_ID	Cloudflare account id. Exactly 32 hexadecimal characters.
R2_ACCESS_KEY_ID	R2 access key. Exactly 32 characters, letters and digits.
R2_SECRET_ACCESS_KEY	R2 secret. Exactly 64 characters, letters and digits.
R2_BUCKET_NAME	flipdeck
R2_PUBLIC_URL	Public base address files are served from.

The quote trap

This single mistake caused three separate debugging sessions on this project. It is worth understanding once.

In a `.env` file, values are conventionally written inside quote marks, and the file format strips them when read. Vercel's settings box does not. It stores exactly what is pasted, quote marks included.

```
In .env this is correct: R2_BUCKET_NAME="flipdeck"
In Vercel this is correct: flipdeck
In Vercel this is broken: "flipdeck"
```

A quoted value is not empty, so it passes any check for a missing setting, then fails much later as something apparently unrelated: a database URL rejected for not starting with the right protocol, an API key that no longer matches its own prefix, a storage address that cannot be resolved.

What was done about it.

All configuration is now read through one helper that strips surrounding quotes, so a quoted value in Vercel no longer breaks anything. Ten tests cover the cases. When pasting into Vercel, still paste the bare value.

Deploying a change

Pushing to the main branch on GitHub deploys automatically. There is nothing else to run.

Environment variables apply at build time, not run time.

A deployment can only see settings that existed when it was built. Saving a new value in Vercel changes nothing on the running site until a fresh build happens. After changing any setting, redeploy: Deployments, then the newest one, then Redeploy.

Running a database migration

The database holds live business data. Migrations are reviewed before they run and applied deliberately, never automatically as part of a deployment.

Order	Step
1	Review the SQL file by hand. Confirm it is additive, or that any destructive step runs only after a backfill.
2	Check the live data it will touch, read-only, before changing anything.
3	Apply it with prisma migrate deploy.
4	Verify the result against the live database.
5	Only then push the code that depends on the new shape.

Migrate first, then deploy.

Additive schema changes are backward compatible, so the old code keeps working against the new database. The reverse is not true: new code reading a column that does not exist yet fails immediately.

Adding a user

Access is invite-only. Signing in with an unrecognised address does not create an account and does not send an email, and the screen gives the same response either way so it cannot be used to discover who has access.

To add someone: Settings, then Users, then add them with a name, email address and role. They can then request a sign-in link.

Troubleshooting

Failures seen on this project, with what each actually meant.

Uploads

Message	Cause and fix
File storage is not configured, missing R2_...	Those settings are absent from the live deployment. Add them in Vercel and redeploy. The message names the specific ones.
R2_ACCOUNT_ID is malformed	The value is present but the wrong shape, usually quote marks or wrong text. Retype it as 32 plain hexadecimal characters.
Please use AWS4-HMAC-SHA256	The request reached Cloudflare with no valid signature. The access key or secret is wrong. Correct lengths are 32 and 64 characters.
That file is N MB, the limit is 4 MB	Applies to PDFs. Photos are automatically reduced before sending and will not hit this.

Sign-in

Symptom	Cause and fix
No email arrives	Either the address has no account, or the Resend key is wrong. Check Settings, Users.
Server error after clicking the link	AUTH_SECRET missing from the deployment.
The link works but the login screen reappears	Was a bug: the page did not redirect an already-authenticated user. Fixed.

Deployment

Symptom	Cause and fix
Build fails on dependency conflicts	The .npmrc file in the repository resolves this. It must stay committed.
Build fails: URL must start with postgresql://	DATABASE_URL has quote marks around it in Vercel. Remove them.
A setting was changed but nothing happened	No rebuild has occurred. Redeploy.

A diagnostic lesson worth keeping

When the browser blocks a request to another service, it deliberately hides the reason. A missing permission rule, an ad blocker, a network filter and a wrong address all produce the same empty error. Uploads were therefore moved to run through the application's own server rather than the browser contacting Cloudflare directly. Failures now return a real status code and a readable message.

PART 3

User Guide

How to use Flipdeck day to day, screen by screen.

Signing in

Go to the site and enter your email address. A sign-in link is emailed to you; clicking it signs you in. There is no password to remember or lose.

If no email arrives, your address probably has no account yet. Ask an Owner to add you.

What each role can see

Section	Owner	Partner	Partner (Lender)	Bookkeeper
Portfolio	Yes	Yes	Yes	Yes
Properties	Yes	Yes	Yes	Yes
Expenses	Yes	Yes	No	Yes
Money In & Out	Yes	No	No	Yes
Payroll	Yes	No	No	Yes
Partners / Company Value	Yes	Yes	Yes	No
Settings	Yes	Own account	Own account	Own account

Bookkeepers see payroll but not capital. Partners see capital but not payroll. This is intentional.

Properties

Creating one. Properties, then Create Property. Address, type, beds, baths, square footage, lot size and storeys are required. A new property arrives with the full budget checklist already in place, every line at zero.

Editing. Open a property and use Edit to change details, status, photo, market value or monthly rent.

Status. Scouting, Under Contract, In Rehab, Rented, Listed, In Escrow, Sold. Rented is the buy-and-hold path: kept and let out rather than sold. Sold removes the property from company assets, because the money has become cash.

Deleting. You are shown exactly what will be removed with it, expenses, payroll and budget lines included, before you confirm.

Budget

The Budget tab lists every subcategory with an Estimated and an Actual column.

- **Estimated** is yours to set. Click and type.
- **Actual** is calculated from the expense log for most categories and cannot be typed in. If it reads zero, no expense has been logged against that line yet.
- **Purchase Costs and Sale Price** are the exception. Those come from a closing statement, so you enter them directly.
- **Labor (Payroll)** appears automatically under Rehab Costs and totals your payroll entries. It cannot be edited here; change the payroll entries instead.

Expenses

The Expenses page is the log of everything spent. Each entry needs a date, property, subcategory, amount and description, and can carry a status of Pending, Paid or Reimbursed, plus a receipt.

Logging an expense is what makes the matching budget line's Actual figure move. That is the only way most Actual figures change.

Money In and Out

This is where bank activity lives. Every transaction belongs to a bank account and has a direction.

- **Money Out** posts to a property as an expense. It needs a property and a subcategory.
- **Money In** posts to a property as income: rent, sale proceeds, a refund, or other.
- **Balances are calculated** from opening balance plus money in minus money out. You cannot type a balance, only the opening figure, so the number always matches the transactions.

Importing a bank statement

Step	What to do
1	Download the CSV from your bank.
2	Import CSV, then choose which account it belongs to.
3	If the columns are not recognised, point each field at the right column. Sample rows are shown to help.
4	Review the preview. Check the Direction column carefully before importing.
5	Import. Rows arrive unassigned; assign a property and category, then Post.

Always check the Direction column.

Banks disagree about signs: some write charges as positive numbers, others as negative. The importer works out which, but on an unusual export a wrong guess would reverse deposits and withdrawals. Rows already imported are detected as duplicates, so re-importing the same statement is safe.

Payroll and workers

Add workers once under Payroll, then Workers. Payroll entries record date, property, worker, hours, rate and notes. The total flows into each property's Labor (Payroll) budget line automatically. A worker with payroll entries cannot be deleted, so history is never orphaned.

Partners and capital

Partners hold capital in three forms:

- **Equity**, money put in as ownership capital.
- **Draw**, money taken back out, subtracted from equity.
- **Loan**, money lent to the company. This is debt and appears under liabilities.

Company Value

The balance sheet for the business as a whole.

Figure	How it is worked out
Total Assets	Value of held properties plus cash on hand. Sold properties are excluded because their proceeds are already counted as cash.
Property value	Market value where you have set one; otherwise the budget's estimated sale price, marked as an estimate.
Cash on hand	Every bank account's opening balance plus money in minus money out.
Total Liabilities	Outside debt plus partner loans, listed separately.
Company Equity	Assets minus liabilities. What the company is worth.
Rental Income	Monthly rent across properties marked Rented only.

Settings

Company details and logo are Owner-only. User accounts are managed under Settings, then Users: add someone with a name, email and role, change a role later, or remove access. Everyone can edit their own name.

Flipdeck was built for Foundational Real Estate. This document reflects the system as of 22 July 2026, covering 38 commits and 7 database migrations.